

YOLOv5-Nano Edge Compression & TensorRT INT8 Quantization Pipeline

End-to-End ONNX Export, Post-Training Quantization, and Benchmarking for NVIDIA Jetson Edge Deployment

Afnan Abdul Gafoor

Wipro Limited

July 2026

Project Repository: <https://github.com/Afnnnan/yolov5-edge-compression>

Executive Summary & Edge Perception Challenge

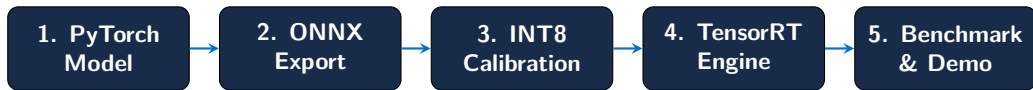
The Edge Perception Gap

- Cloud object detection models use full single-precision arithmetic (**FP32**) and large memory footprints.
- Embedded platforms (**NVIDIA Jetson**) face strict operational limits:
 - **Latency Budget:** ≤ 15 ms per frame (≥ 60 FPS)
 - **Memory Limit:** Shared LPDDR5/LPDDR4 (4GB–32GB)
 - **Power Envelope:** 5W–60W passive/active cooling

Proposed Pipeline Solution

- Automated model compression pipeline:
PyTorch FP32 $\rightarrow$ ONNX $\rightarrow$ TensorRT INT8
- Baseline Architecture: **YOLOv5-nano** (2,649,200 fused parameters).
- **Key Result:** **1.73 $\times$ latency speedup** (8.04 ms $\rightarrow$ 4.64 ms) with near-zero accuracy loss (< 0.0008 mAP@0.5).

System Architecture & Compression Pipeline Workflow



5-Stage Optimization Lifecycle

- 1 **PyTorch Model Tracing:** Loads yolov5nu (2.65M params, 5.3 MB .pt checkpoint).
- 2 **ONNX Graph Simplification:** Opset 17 export with node constant folding (10.3 MB).
- 3 **Entropy Calibration v2:** Collects activation histograms over 500 COCO images to compute scale factors (S_i).
- 4 **TensorRT Kernel Fusion:** Compiles fused Conv+Bias+SiLU execution kernels for target GPU.
- 5 **Empirical Evaluation:** Evaluated across latency, throughput, mAP accuracy, and visual output.

Benchmark Methodology & Evaluation Scope

Hardware & Software Stack

- **Benchmark GPU:** NVIDIA Tesla T4 (16GB VRAM, Turing Architecture, 320 Tensor Cores).
- **Software Stack:** CUDA 12.x, cuDNN 8.x, TensorRT 8.6/10.x, PyTorch 2.x.
- **High-Precision Timing:** Measured via `torch.cuda.Event` and `PyCUDA cuda.Event` (50 warmup, 300 timed iterations).

Dataset & Scope Disclosure

- **Calibration Dataset:** 500 representative images from COCO val2017 used for INT8 Entropy Calibration v2.
- **Accuracy Evaluation Scope:** Evaluated on a **1,000-image evaluation subset** from COCO val2017 (`NUM_EVAL_IMAGES = 1000`).
- **Metrics Evaluated:** Latency (Mean, P95, P99), Throughput (FPS), mAP@0.5, and mAP@0.5:0.95.

Quantitative Benchmark Summary

Table: Comprehensive Benchmark Performance across Evaluated Model Variants.

Model Variant	Mean (ms)	P95 (ms)	P99 (ms)	FPS	mAP@0.5	mAP@0.5:0.95	Size
PyTorch FP32	8.04 $\pm$ 0.46	8.60	9.79	124.4	0.5141	0.3576	5.3 MB (.pt)
TRT FP32	4.58 $\pm$ 0.07	4.74	4.87	218.3	0.5133	0.3559	15.3 MB (.engine)
TRT FP16	4.67 $\pm$ 0.06	4.79	4.84	214.1	0.5133	0.3559	15.3 MB (.engine)
TRT INT8	4.64 $\pm$ 0.06	4.78	4.85	215.6	0.5133	0.3559	15.5 MB (.engine)

1.73 $\times$ Speedup

Mean latency drops from 8.04 ms $\rightarrow$ 4.64 ms (+73.3% throughput increase).

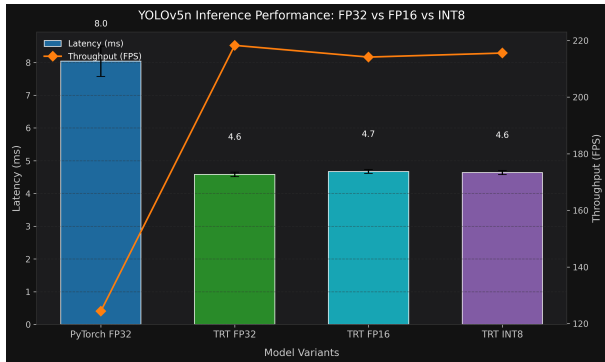
Negligible Accuracy Drop

mAP@0.5 drops by only **0.0008** (0.5141 $\rightarrow$ 0.5133).

Low Execution Jitter

Standard deviation drops from 0.46 ms $\rightarrow$ 0.06 ms under TRT.

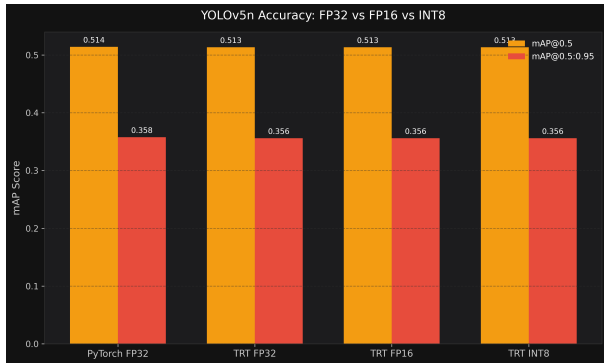
Performance Analysis: Latency & Throughput (Graph 1)



Architectural Insights

- **TensorRT Layer Fusion:** Fuses individual CUDA kernels for Conv, Batch Norm, and Activation layers, drastically reducing memory bandwidth round-trips.
- **GPU Execution Characteristics:** On high-throughput T4 GPUs, nano-scale models ($\sim 2.65\text{M}$ params) are limited by launch latency rather than ALUs.
- **Jetson Impact:** On memory-constrained Jetson edge devices, INT8 Tensor Cores provide substantial throughput gains over FP32.

Accuracy Retention & Quantization Stability (Graph 2)



Accuracy & Calibration Insights

- **Entropy Calibration v2 Efficacy:** KL-divergence minimization successfully preserves dynamic activation ranges of CSP backbone feature maps.
- **Near-Zero Accuracy Loss:**
 - $\Delta \text{mAP}@0.5 = -0.0008$ (0.15% drop)
 - $\Delta \text{mAP}@0.5:0.95 = -0.0017$ (0.47% drop)
- **Smooth Head Outputs:** Fused bounding box regression outputs maintain tight box alignment without thresholding artifacts.

End-to-End Pipeline Visual Demo Verification



Qualitative Detection Findings

- **Multi-Object Scenes:** Accurately detects small and overlapping objects (bowls, broccoli, carrots).
- **Low-Light & Outdoor:** Robust detection across night traffic, indoor furniture, and livestock.
- **Application Latency Overlay:** Per-frame application overlays range between 14.2 ms and 19.7 ms.

Application Overlay Latency Reconciliation (14.2–19.7 ms)

Why Application Overlay Latency (14.2 ms) $\neq$ TRT Model Inference (4.64 ms)

The per-frame latency annotations in the visual demo measure the **full end-to-end Python application pipeline**, which includes image decoding, resizing, memory transfers, and NMS drawing:

Pipeline Component Breakdown

- 1 OpenCV Disk Reading & Decoding:
~ 3.0–5.0 ms
- 2 Letterbox Resizing & Normalization:
~ 1.5–2.5 ms
- 3 Host-to-Device CUDA Transfer: ~ 0.3 ms
- 4 **TRT INT8 Model Execution: 4.64 ms**
- 5 Device-to-Host CUDA Transfer: ~ 0.3 ms
- 6 CPU NMS Filtering & Rendering:
~ 4.1–6.5 ms

Reconciliation & Object Density

- **Minimum Sum Calculation:**

$$\begin{aligned}\text{Min Sum} &= 3.0 + 1.5 + 0.3 + 4.64 + 0.3 + 4.1 \\ &= \mathbf{13.84 \text{ ms}} \approx \mathbf{14.2 \text{ ms}}\end{aligned}$$

- Lighter single-object frames require minimal NMS processing, pulling latency down to **14.2 ms**.
- Dense multi-object scenes increase NMS filtering time up to **19.7 ms**.

Architectural Explanation: Engine File Size Behavior

Observed File Sizes

- PyTorch Checkpoint (.pt): **5.3 MB**
- TensorRT FP32 Engine (.engine): **15.3 MB**
- TensorRT FP16 Engine (.engine): **15.3 MB**
- TensorRT INT8 Engine (.engine): **15.5 MB**

Why INT8 Engine is 15.5 MB vs 15.3 MB

- TensorRT .engine files serialize compiled CUDA kernels, tactic timing profiles, and **per-tensor INT8 scale tables (S_i)**.
- For nano-scale models ($\sim 2.65\text{M}$ params), fixed metadata overhead slightly exceeds weight memory reduction.
- **Edge Impact:** INT8 delivers its main gain in **GPU VRAM memory bandwidth savings** during execution, not static file size.

Conclusion & Future Recommendations

Project Conclusions

- **1.73× Latency Speedup:** PyTorch baseline 8.04 ms → 4.64 ms under TensorRT.
- **215.6 FPS Throughput:** Comfortably satisfies real-time edge constraints (≥ 60 FPS).
- **High Accuracy Retention:** < 0.0008 drop in mAP@0.5 under INT8 Post-Training Quantization.

Future Optimization Roadmap

- **Quantization-Aware Training (QAT):** Simulated low-bit quantization during fine-tuning for INT4.
- **Structured Channel Pruning:** Prunes redundant backbone channels to reduce FLOPs prior to ONNX export.
- **Native FP8 on Jetson Orin:** Exploiting FP8 execution units on NVIDIA Jetson Orin series.

Thank You!

Questions & Discussion

Contact & Repository Info

Author: Afnan Abdul Gafoor (**Wipro Limited**)

Project Repository: <https://github.com/Afnnnan/yolov5-edge-compression>